Programmable In-Network Timing Obfuscation for DNP3

Philip Akekudaga
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A.
philip.akekudaga@uri.edu

Hui Lin
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A.
huilin@uri.edu

***Abstract*—Device fingerprinting is an essential step in cyber reconnaissance against industrial control systems (ICSs). In a power grid, a passive observer on the control network can tell the model of a DNP3 outstation and the type of operation it performs just from the timing of its responses, and this timing stays visible even when the payload is encrypted. Because the traffic obfuscation defenses built for general web traffic pad, split, and delay packets on the end host, they do not directly transfer to ICS traffic, since a protocol like DNP3 validates every block of its payload and outstation devices such as protective relays cannot be modified to shape their own traffic. In this paper, we present an in-network timing obfuscation framework that runs in the data plane of a single programmable switch at the outstation edge. The framework classifies each DNP3 transaction, holds the packets that carry the outstation's timing, and releases them at configurable offsets, so that the timing the observer records is a policy value rather than the device's own. We use the two timing features introduced by Formby et al. as case studies: the cross-layer response time (CLRT) of READ and of the SELECT phase of select-before-operate control, and the master-visible timing of an OPERATE. We implement the framework as a P4 program on an Intel Tofino-1 and evaluate it against a physical SEL-751A relay over 22 independent sessions and 63,360 transactions. With the timing mechanism off, the median CLRT is 2.116 ms for READ, 2.050 ms for SELECT and 2.937 ms for OPERATE, each with an interquartile range near 2.7 ms; with it on, all three settle at 4.000 ms with an interquartile range of 0.006 ms, in every session. Evaluating on sessions the classifier never saw, the balanced accuracy of a three-class transaction classifier falls from 0.651 to 0.333, which is chance, and the mutual information between the class and the CLRT falls from 0.266 bits to 0.007 bits. The framework adds about 21 to 23 ms to a transaction and adds no packet and no byte, and a sweep of its two offsets shows that the delay it costs and the interval an observer sees can be chosen independently. These results show the suppression of one timing feature on one relay; they do not hide the function codes that remain visible, they leave the acknowledgment latency of the control path distinguishable from that of the read path, and they do not show indistinguishability across devices.**

***Index Terms*—traffic obfuscation, device fingerprinting, DNP3, programmable switches.**

I. Introduction

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies present network traffic obfuscation [3], [4]. Because device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5], traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods do not directly transfer device fingerprinting in ICS/SCADA environments for two main reasons. First, the leaked features are different. General web traffic obfuscation usually targets the flow level sizes and timing patterns [7] whereas ICS device fingerprinting is carried by the response timing of the protocol exchanges, which stays visible even when the payload is encrypted [10]. Secondly, ICS protocols leave no room or flexibility to add bytes or dummy or fabricated packets. This is because a protocol like DNP3 validates its payloads, so padding and injected packets will break the exchange when the master reassembles the response packets and checks the cyclic redundancy check (CRC) for every sixteen application bytes [11]. Consequently, a defense in this setting must hide the timing that leaks even after encryption while changing no bytes.

In this paper, we present an in-network mechanism that normalizes an outstation's fingerprintable features using the data plane of the Tofino programmable switch at the outstation edge, without changing the endpoints or modifying the

bytes of the DNP3 traffic. This defense holds per transaction responses and releases them at configurable offsets so the master-visible timing then becomes a policy value. We implement this as a P4 program on a single switch and evaluate it on a cyberphysical testbed and make the following contributions:

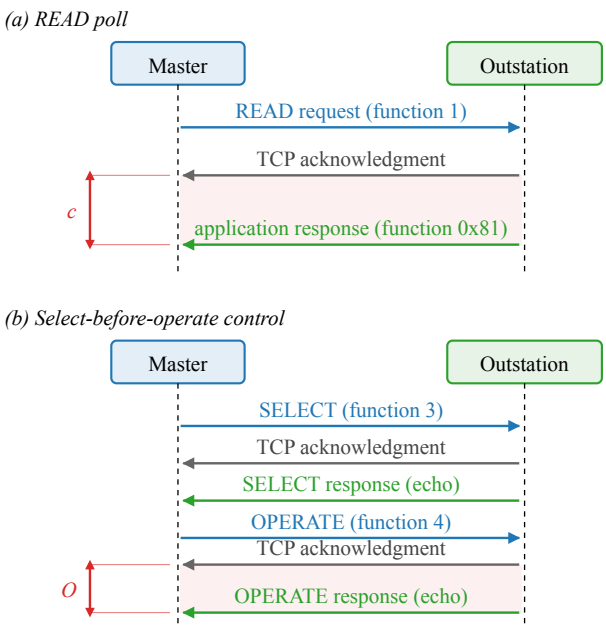
- We propose an in-network turning framework that obfuscates timing features and shapes how an on-path passive observer fingerprints outstation devices. This, to the best of our knowledge, is the first in-network defense for ICS device-fingerprinting features.
- A case study of the timing frameworks introduced by Formby et al [10], we present a multi-configurable mode of operation that holds TCP acknowledgments and DNP3 Responses and releases at predetermined offsets from the request, such that the master visible response time is deterministic
- Normalizes the control command and operation transaction time. As a second case study covering the physical fingerprinting feature of Formby et al. We build a hold mechanism to hold and determine the observed time on the master-facing observed traffic and show the passive observer records a time that is obfuscated
- We evaluate this mechanism on a single physical Tofino and Outstation device.

II. Background and Motivation

In this section we focus on the DNP3 transactions the paper covers, the features that leak from them and how an adversary uses them in fingerprinting, and the opportunity that a programmable data plane gives us to change the timing without touching the endpoints.

The DNP3 read and control transactions. DNP3 carries supervisory control and data acquisition (SCADA) traffic over TCP between a master and an outstation [11]. The master polls the outstation with a READ request (function code 1), and the outstation returns the requested measurements in an application-layer response (function code 0x81). A control command, on the other hand, uses select-before-operate (SBO): the master first sends a SELECT (function code 3) that names an output point, and the outstation responds by echoing the request; the master then sends an OPERATE (function code 4), which the outstation confirms with a response that echoes the command. Because DNP3 runs over TCP, every request is also acknowledged at the transport layer before the application answers. As such, each transaction on the wire is a request, a TCP acknowledgment that carries no DNP3 payload, and a DNP3 response, in that order, as Figure 1 shows. We use these terms consistently in the rest of the paper: the acknowledgment is the TCP segment with the ACK flag that the outstation returns first, and the response is the DNP3 application frame that follows it.

Why the transaction timing is a fingerprint. The acknowledgment is produced by the outstation's TCP stack as soon as the request arrives, whereas the response is produced by its application only after the device has done the work the request asks for. Consequently, the interval between the two,



c: acknowledgment-to-response interval of the READ (CLRT).
O: master-visible OPERATE ACK-to-echo interval.

Fig. 1. The DNP3 transactions the paper measures, as seen on the master-facing link; requests in blue, transport-layer acknowledgments in grey, and application responses in green. (a) A READ poll: the request, the outstation's transport-layer acknowledgment, and its application-layer response. The interval *c* between the acknowledgment and the response is the cross-layer response time (CLRT). (b) A select-before-operate control: SELECT and OPERATE each produce an acknowledgment and a response that echoes the command. The interval *O* between the acknowledgment and the echo of the OPERATE is the master-visible OPERATE ACK-to-echo interval.

the CLRT, reflects the outstation's internal processing rather than the network path, and it differs between device models and between the kinds of work they do. Formby et al. used it to tell device types and models apart in a substation network, and used the time a device takes to complete a control operation as a second feature that reveals the physical device behind the command [10]. Our own measurements on a physical SEL-751A relay show the same effect on one device: with no timing defense, the median CLRT is 2.116 ms for a READ, 2.050 ms for a SELECT and 2.937 ms for an OPERATE, each with a long tail, so the three transaction classes are already partly separable from timing alone (Section VI). The function codes remain visible in the plaintext, so the classes themselves are not a secret; what the timing adds is a channel that identifies the device and its operation without any payload, and that survives encryption.

Measured and inferred events. Everything the paper measures is a packet timestamp on the master-facing link, i.e., the request, the acknowledgment, and the response. The switch-internal release times and any event on the relay-facing link are not observed; where the paper names them, they are inferred from the configuration. Physical breaker motion is never measured.

Programmable data planes. A programmable switch ex-

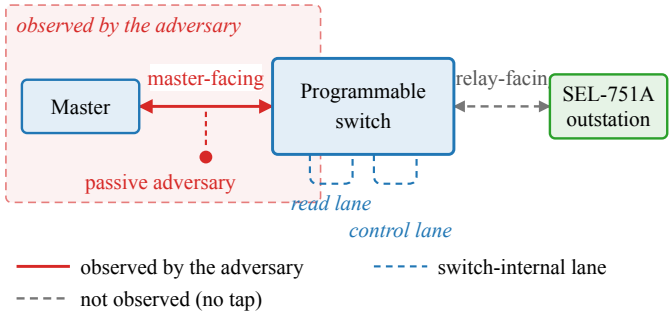


Fig. 2. Observation model. The master reaches the SEL-751A outstation through one programmable switch. The passive adversary sees only the master-facing link (solid red, shaded region); the relay-facing link and the two switch-internal scheduling lanes (dashed) are not observed, by the adversary or by our measurements.

poses a match-action pipeline, written in P4 [12], that parses headers, keeps per-packet state in registers, and places packets into output queues at line rate. Because such a pipeline can delay and release a packet without a host in the loop, it can hold an outstation's acknowledgment and response and release them on a schedule that the operator sets, while changing no endpoint and no byte of the payload.

III. Threat Model and Objectives

Assumptions on the adversary. The adversary is a passive observer on the master-facing link and has the ability to record TCP and IP headers, segment sizes, and arrival times, but neither modifies nor injects packets, as in the fingerprinting attack of Formby et al. [10]. Figure 2 shows the setup. Because DNP3 is usually deployed without transport security across shared substation networks [13], we assume the traffic here is unencrypted. Therefore, the adversary is able to read the exchanges directly, including the DNP3 function codes, and from them it derives the transaction timing of Section II and can train a classifier that separates transaction classes from timing features. The adversary cannot compromise the master or the outstation, control the switch, or read the switch's internal state. It also does not observe the relay-facing link, because in our deployment that link is inside the switch and no tap exists, and it cannot see physical breaker motion. An adversary that actively interacts with the traffic, sends its own probes, or changes the network is out of scope; we stay with the passive observer, which is the setting that prior traffic obfuscation work targets.

Reconnaissance target. The adversary's objective is to fingerprint outstation devices and obtain knowledge to prepare for a later attack. Because control-related attacks such as false data injection attacks work only when the attacker knows the target devices [14], reconnaissance is the enabling step to identify the devices and their weaknesses. The fingerprinting process can succeed without decoding the payloads because it is done using the timing features of the master-outstation exchanges. Following [10], the two timings in scope are the cross-layer response time (CLRT), i.e., the interval between the transport-layer acknowledgment and the application-layer response of a READ or of the SELECT phase of SBO, and the master-visible OPERATE ACK-to-echo interval, i.e., the interval between an OPERATE's acknowledgment and its echo. The CLRT reflects the outstation's internal processing, while the second reflects the physical device behind a control command. The adversary also sees the request-to-acknowledgment interval, which we report as measured.

Research objectives. Our objective is therefore to remove these features from the master-facing observing adversary. We state it as three properties in line with our design and evaluation.

- **RO1 (read timing).** The visible CLRT of READ and of the SELECT phase of SBO is a configurable policy value that is independent of the transaction, so it no longer reflects the outstation's original processing signature.
- **RO2 (control timing).** The visible OPERATE ACK-to-echo interval is a fixed policy value that is independent of the switch's internal release delay applied to the command.
- **RO3 (timing leakage).** The measured timing features carry less information about the transaction class, such that a classifier trained on Timing OFF timing features performs no better than chance on Obfuscated ones.

What is out of scope. The framework does not hide the function codes, which stay visible in plaintext DNP3; RO3 is about the timing channel, not the command semantics. It does not change packet sizes, and no size-related claim is made in this paper. Additionally, it does not claim that two different devices become indistinguishable: the evaluation has one outstation, and what it shows is that this outstation's timing signature is replaced by a policy signature.

IV. Design

In Figure 3, we show the design of the hold and timing obfuscation mechanism. When a request from the master enters the switch, the ingress classifies it by its function code, records the state it needs to recognize the outstation's answer, and forwards the request through the pipeline without a delay. When the outstation answers, the answer goes through the same pipeline, but this time the program holds the acknowledgment and the response in switch queues in the traffic manager and releases each one at a predetermined offset. In this way, the master sees a timing value that the policy sets instead of the timing that the outstation itself produces, which is the signature used for fingerprinting. Because the read and the control responses must not interfere with each other, we implement a separate hold lane for each, and because the two transactions expose different information, each lane uses its own anchor for the offsets. The rest of the section presents the mechanisms and what they require.

Classification and transaction state. The switch parses the DNP3 link and application headers of every packet on the protected session and classifies each packet as a request

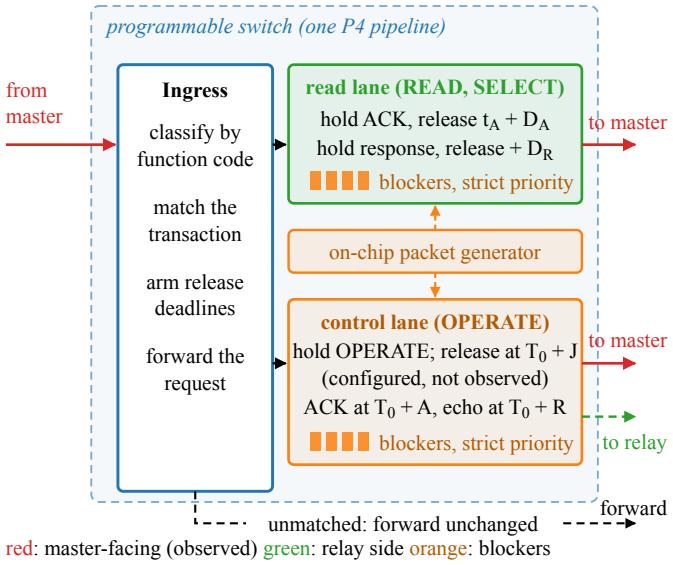


Fig. 3. Design of the hold and timing obfuscation mechanism. Ingress classifies each DNP3 transaction and arms its release deadlines. The read lane holds the acknowledgment and the response of a READ or SELECT and releases them relative to the outstation's acknowledgment. The control lane holds an OPERATE, releases it toward the relay at a configured delay that is not observed, and releases its acknowledgment and echo relative to the request. The generator fills the blocker reservoirs (orange); a strict-priority scheduler drains them first, which realizes every deadline. Red arrows are master-facing and observed; the green dashed arrow is the configured relay-facing release at $T_0 + J$, not observed. Unmatched traffic is forwarded unchanged.

(READ, SELECT, or OPERATE, by function code), an acknowledgment (the outstation's first segment with the TCP ACK flag after a request), a response (a DNP3 application frame from the outstation), or other. A small set of registers holds the state of the one transaction in flight on the session, i.e., an active-transaction tag, the expected acknowledgment number of the outstation's reply, the armed deadlines, and, for control, the identity of the held OPERATE. The state is written once per event and read by the packets that follow, so a duplicate acknowledgment or a retransmitted request cannot arm a second deadline.

Anchoring the read deadlines to the acknowledgment. A held packet must leave the switch at a specified deadline, but the switch and P4 have no general-purpose timer. For a READ or a SELECT, the feature to remove is the CLRT, the interval between the acknowledgment and the response. Therefore, when the outstation's acknowledgment reaches the switch at time t_A, the program arms two deadlines from the policy offsets D_A and D_R,

$$t_{\text{ack}} = t_A + D_A, \qquad t_{\text{resp}} = t_A + D_A + D_R, \tag{1}$$

at which it releases the TCP acknowledgment and the DNP3 response to the master respectively. Consequently, the CLRT interval the adversary measures is

$$\text{CLRT} = t_{\text{resp}} - t_{\text{ack}} = D_R, \tag{2}$$

a policy value that replaces the CLRT interval that the outstation's internal processing produces and that serves as its fingerprint. Because the switch can delay a packet but cannot release it before it exists, equation (2) holds only when the response has arrived by $t_A + D_A + D_R$; as such, $D_A + D_R$ must be larger than the largest CLRT the outstation produces on its own for the traffic to be defended, and every response in our captures met that deadline. Note that the request-to-acknowledgment interval then becomes the outstation's own acknowledgment latency plus D_A: it is shifted by a constant, but the sub-millisecond spread of the outstation's TCP stack remains in it.

Anchoring the control deadlines to the request. A control command is different, because the framework also holds the OPERATE itself before it reaches the relay. This separates the moment the master sent the command from the moment the relay acts on it, which an operator may want for a control command; but the outstation's acknowledgment then exists only after the release, so a deadline of the form $t_A + D$, as in (1), would carry the length of the hold into the observable. Therefore, the control path is anchored to the request instead. When the OPERATE arrives at time T_0, the switch holds it and releases it to the relay at an internal delay J, while the acknowledgment and the echo the master sees are placed at fixed offsets from T_0,

$$t_{\text{ack}} = T_0 + A, \qquad t_{\text{echo}} = T_0 + R, \tag{3}$$

where A and R are configured such that A exceeds J plus the outstation's acknowledgment latency, which the control plane checks before it installs the values. The master-visible OPERATE ACK-to-echo interval is therefore

$$O = t_{\text{echo}} - t_{\text{ack}} = R - A, \tag{4}$$

in which J does not appear. J is drawn per transaction from a configured set, so the relay-facing release time varies while the master-facing observable does not. In our experiments $D_R = R - A = 4$ ms, so both case studies present the same 4 ms policy value to the observer.

A family of release policies. The two offsets are independent, and the choice of which of them is non-zero names a policy. Setting $D_A = 0$ releases the acknowledgment at once and holds only the response, a response-only policy. Setting $D_R = 0$ holds the acknowledgment and lets the response follow it immediately, an acknowledgment-only policy. Setting both above zero holds each packet to its own deadline, which is the dual-deadline policy we implement and evaluate. A fourth policy would release the held acknowledgment on an event, e.g., the arrival of the matching response, rather than on a deadline; it is driven by an event and is therefore not a parameter case of the other three, and we do not implement it. Because the response-only and acknowledgment-only policies are limits of one mechanism, a single datapath covers the family, and an operator selects a point in it by choosing the two offsets rather than by loading a different program.

Choosing the budget. The switch can release only a packet it is already holding, so the budget $D = D_A + D_R$ has to exceed the outstation's own response time. Consequently D is both the coverage knob and the delay the framework adds,

and choosing it is a trade-off rather than a free parameter. The observer, on the other hand, sees only D_R, because by (2) the visible interval is the gap between the two releases. Since the budget and the observable are set by different quantities, an operator can hold the cost fixed and still choose what the timing feature looks like. A second bound comes from the other direction: the reservoir that starves the queue carries a finite pass budget, so a hold cannot outlast the fail-open horizon H after which the packet is released regardless. The usable region is therefore D above the outstation's response-time tail and below H, and Section VI-C measures both edges.

Security and deadlines. The anchor matters as much as the offsets. On the read path the request is forwarded at once and only the reply is held, so anchoring to the outstation's acknowledgment costs nothing and lets the hold start as late as possible; by (2), the CLRT becomes D_R regardless of how long the outstation took. On the control path the command is held, so the acknowledgment is a function of J, and only a request-anchored rule keeps J out of the observable, by (4). As a result, an adversary that observes and measures either interval learns the policy and not the physical device behind the transaction, and because J is not present in (4), the adversary cannot subtract a known offset to recover the device's original processing time either.

Reservoir tokens as data-plane timers. As the programmable switch has no timer, to achieve a deadline from scheduled packets alone, a held packet must wait in its queue behind a reservoir of blocker packets that sit in a higher-priority queue on the same port. A strict-priority scheduler in the traffic manager drains the blockers first, so the held packet cannot leave until the reservoir is empty. If τ is the time to drain one blocker, a reservoir of K blockers will hold a packet for about $K\tau$, and the control plane sizes K for each deadline. The blockers are generated by the switch's on-chip packet generator when the request arrives, they circulate on internal ports, and they are dropped in the switch when they expire. Consequently, they never leave the switch and are not visible to the master nor the outstation. A tag in each blocker names the transaction and the lane it belongs to, so a stale blocker from an earlier transaction cannot delay a later one.

Independent scheduling lanes. A read response and a control response can be held at the same time, and if they were serialized in a single lane, whichever holds longer would delay the other and hence reshape the timing policy that is fixed. This therefore gives each treatment its own internal recirculation lane and its own traffic manager queues, which are independent of each other, and the register state of the two lanes is likewise separate.

Unmatched transactions and failing open. A defense on a control path must not become an availability risk. As such, a packet that does not belong to a protected session, a request the framework does not recognize, an acknowledgment with no armed transaction, or a response that arrives after its transaction has retired is forwarded unchanged. In this way, an unexpected exchange loses its protection but not its delivery. Every hold is also bounded: a fail-open budget on the number of blocker passes releases a held packet if its reservoir has not drained within a configured horizon and clears the transaction state, so a lost acknowledgment or response cannot leave a deadline armed forever. With the timing mode switched off, the same program forwards every packet immediately and arms nothing.

Byte preservation. The framework only holds and forwards packets. Since it neither pads a response, injects a packet into the protected exchange, nor edits a DNP3 field, every byte the master reassembles is the byte the relay produced, and the payload's cyclic redundancy checks remain valid.

V. Implementation

Target and program. We implement the design as a single $\mathrm{P4}_{16}$ program for the Tofino Native Architecture on an Intel Tofino-1 switch, compiled with the Barefoot SDE 9.13 compiler. The program occupies all twelve ingress match-action stages of one pipeline and six egress stages, with 112 tables. The classification of Section IV is a parser for the Ethernet, IP, TCP, and DNP3 link and application headers followed by a chain of match-action tables, and the transaction state is a set of one-entry registers accessed by stateful ALU actions, one access per packet per register. The final forwarding decision is made by one table keyed on a compact outcome computed by the earlier stages, so that every disposition (forward, hold, release, or drop an expired blocker) is an explicit entry.

Ports and lanes. The master is cabled to one front-panel port and the relay to another; these are the only external links. Two further ports are configured in loopback and carry no cable: one is the read lane and the other the control lane of Section IV, each with its own hold and blocker queues at descending strict priority.

Timing state. The deadlines are 32-bit words in units of 256 ns taken from the ingress timestamp of the anchoring packet. On the read path the acknowledgment's arrival is the anchor and the offsets D_A and $D_A + D_R$ are added to it. On the control path the OPERATE's arrival is the anchor and A, R, and J are added to it, and the acknowledgment and echo deadlines are written when the OPERATE is admitted, so that the outstation's later acknowledgment cannot re-anchor them. J is selected per transaction by a table keyed on a data-plane random value from a configured set of admissible holds. The offsets D_A, D_R, A, R, the J set, the fail-open budget, and the timing mode (off or on) are all runtime table entries.

Control plane. A Python control plane over the switch's runtime interface brings up the ports, creates the queues, installs the multicast and generator configuration, writes the timing parameters, and reads every value back. The values are checked before they are installed: the offsets must be multiples of the tick, A must exceed the largest J plus the outstation's acknowledgment latency, and R must exceed A. A failed check leaves the mechanism disabled rather than partly configured. The guarded test harness that issues a physical SELECT and OPERATE permits only two isolated control points on the relay and refuses the breaker-close point.

Which policies the build realizes. The program defines the release policies of Section IV as mode values, but the disposition tables that seed the blocker reservoir carry entries only for the bypass path and for the dual-deadline policy. Because a mode with no entry never seeds a reservoir, it never arms a hold, so the loaded build realizes those two and no other, even though the remaining mode values stay installable from the control plane. This is a consequence of fitting the pipeline: collapsing the disposition logic into two priority-ordered ternary tables is what brought the program within the twelve ingress stages, and the separate gates for the other policies did not survive that collapse. We therefore evaluate the dual-deadline policy, and the response-only and acknowledgment-only policies appear in Section VI-C as limits of its two offsets rather than as run-time selections.

What ran, exactly. The evidence in Section VI was produced by one program loaded once, whose source and compiled binary are identified by hash in the repository. That program is a combined implementation: besides the timing mechanism described here, it carries a size-shaping datapath that we developed separately and that can be enabled independently of the timing mode. For the campaign reported in this paper the size datapath is disabled in both arms, which we verified by control-plane readback and on the wire, where every response arrives as one application-layer segment and every capture carries the same frame and byte counts in both arms. The two arms therefore differ only in the release schedule. We name them *Timing OFF* and *Obfuscated* rather than native and defended, because the Timing OFF arm is the same binary with the release schedule bypassed and not an untouched relay. This paper evaluates timing only; the size datapath is not evaluated, claimed, or figured.

VI. Evaluation

We evaluate the framework on a physical relay against the three research objectives of Section III. Because a single capture session cannot show that a release schedule holds in general, the evaluation is built on a campaign of independent sessions and every classifier result is reported on sessions the classifier never saw.

A. Testbed

Devices and topology. The master is a server running a DNP3 driver, the outstation is a physical SEL-751A feeder relay, and between them sits an Intel Tofino-1 switch running the P4 program of Section V. The master reaches the relay only through the switch, which we confirmed by disabling the relay-facing port and observing that the relay became unreachable. All packet timestamps are taken on the master-facing link, so every interval we report is what a passive observer on the control network would record.

Conditions. The two arms are *Timing OFF*, in which the loaded program forwards the outstation's acknowledgment and response as they arrive, and *Obfuscated*, in which the program holds them and releases them on the deadlines of Section IV. Both arms run the same binary, and in both arms the size carve is disabled, so the only difference between them is the release schedule. We verified the setting on the control plane by readback and on the wire, where every response arrives as one application-layer segment.

Control point. The SELECT and OPERATE transactions address isolated binary output points, i.e., points the relay's own settings map to remote bits carrying no output equation, so every control transaction the relay accepts and echoes drives no contact.

B. Data and Extraction

Campaign. The dataset is 22 sessions collected over a five-hour window. Each session is six blocks of 400 READ and 40 select-before-operate transactions, interleaved with a minimum spacing of six reads between control transactions, with the arm reconfigured between blocks in a balanced and rotated order and the driver seed recorded, e.g., a session alternates the two arms rather than running one arm to completion. In total the campaign contains 63,360 transactions: 26,400 READ, 2,640 SELECT and 2,640 OPERATE in each arm.

Extraction. For each transaction we pair the request with the outstation's first transport-layer acknowledgment and with the DNP3 application response that follows it, and we record three intervals: the acknowledgment latency from request to acknowledgment, the CLRT from acknowledgment to response, and the response time from request to response. No transaction is excluded. Every request was answered, every READ response was well formed, and all 5,280 control transactions returned a success status, so the analysed set is the collected set.

Evaluation protocol. Sessions are the unit of independence. Every classifier result in Section VI-F is obtained by training on 21 sessions and testing on the session held out, repeated over all 22 sessions, so no result is reported on data the classifier was fitted to.

C. Choosing the Offsets

Why the budget is the cost. The switch can release only a packet it already holds, so the budget $D = D_A + D_R$ has to exceed the outstation's own response time, and D is therefore both the coverage knob and the delay the framework adds.

Result. In Figure 4(a, b), we show the fraction of Timing OFF transactions whose CLRT exceeds a given value, and the fraction that a given budget cannot place on the schedule. At $D = 24$ ms, 99.905% of transactions can be scheduled and 0.095% arrive too late to be held. Raising D to 60 ms recovers a further 0.082 percentage points for two and a half times the delay, so the curve is already flat at the operating point. The same fraction appears under the mechanism, where 0.091% of READ transactions depart from 4.000 ms by more than 1 ms, since these are the responses that reached the switch after their scheduled release. The budget therefore predicts its own residual.

The observable and the cost are independent. In Figure 4(c, d), we show the master-facing intervals measured while sweeping the acknowledgment offset, and while holding

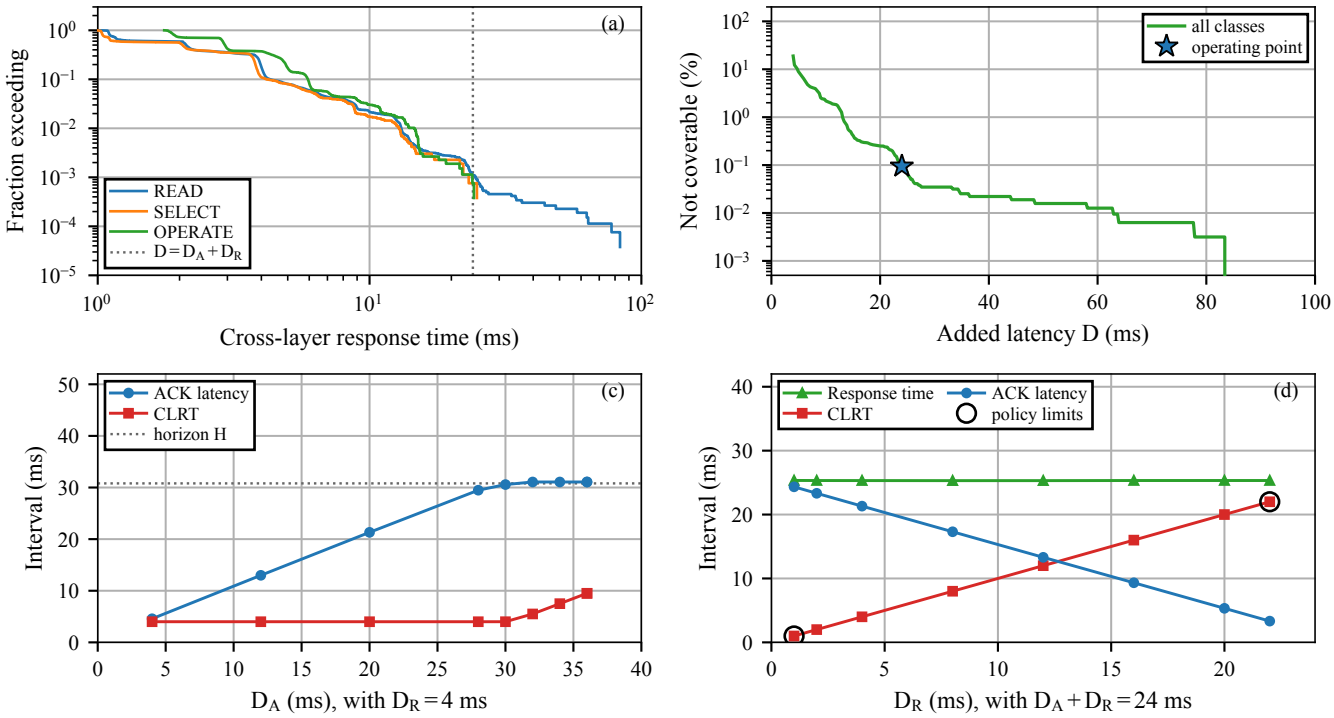


Fig. 4. **Choosing and bounding the two offsets.** (a) fraction of Timing OFF transactions exceeding a given CLRT; (b) transactions not coverable against added latency D; (c) sweep of D_A at $D_R = 4$ ms; (d) fixed budget $D_A + D_R = 24$ ms, split varied.

the budget at 24 ms and moving the split. The CLRT follows D_R from 0.998 ms to 22.001 ms while the response time stays within 25.307 to 25.339 ms. Consequently an operator can choose what the observer sees without changing what the defense costs, and the acknowledgment-dominant and response-dominant policies are the two ends of that choice.

The upper bound. Sweeping D_A at $D_R = 4$ ms, the acknowledgment tracks D_A until it saturates at 31.07 ms, after which the CLRT is no longer pinned, reaching 5.503 ms at $D_A = 32$ ms and 9.507 ms at $D_A = 36$ ms in two independent runs. The saturation is the program's fail-open horizon of 30.8 ms, the safety bound after which a held packet is released regardless, counted from the arrival of the request because the tokens that starve the queue are seeded there. The usable region is therefore bounded below by the tail requirement of about 24 ms and above by that horizon, and the operating point we use gives the tightest release we measured, with a CLRT standard deviation of 0.007 ms.

D. Effectiveness in RO1: READ and SELECT CLRT

Why. RO1 asks whether the read-path rule of (1) pins the CLRT of a READ and of the SELECT phase of SBO to the policy value, independently of what the relay itself did.

Result. In Figure 5, we show the CLRT and the acknowledgment latency for each transaction class and arm. With the mechanism off, the median CLRT is 2.116 ms for READ and 2.050 ms for SELECT, and the interquartile range of each is about 2.7 ms. With the mechanism on, both settle at 4.000 ms and the interquartile range falls to 0.0060 ms, a reduction of 466 and 451 times. This is because the release instant is computed from the outstation's acknowledgment and a fixed offset, so it does not depend on how long the device took to answer.

Distributions. In Figure 6, we show the same measurements as empirical distributions. With the mechanism off the curves rise gradually over several milliseconds and do so in a small number of discrete steps, near 1.05, 1.15 and 2.15 ms for READ, so a single observation is informative about the class. With the mechanism on each curve is a step at 4.000 ms. It is this modal structure, and not the median alone, that a classifier learns, which is why removing it matters more than shifting it.

Stability across sessions. In Figure 7, we show the median CLRT of every session, so that the result is not read from one capture. In all 22 sessions and for all three transaction classes the Obfuscated median is 4.000 ms and the interquartile range stays below 0.01 ms, while the Timing OFF medians remain separated and carry a wide spread. As such, the pinned release is a property of the mechanism rather than of a particular session.

E. Effectiveness in RO2: Master-Visible OPERATE Interval

Why. RO2 asks whether the control-path rule of (3) keeps the master-visible acknowledgment-to-echo interval of an OPERATE at a fixed policy value while the relay-facing release varies over the configured hold codebook.

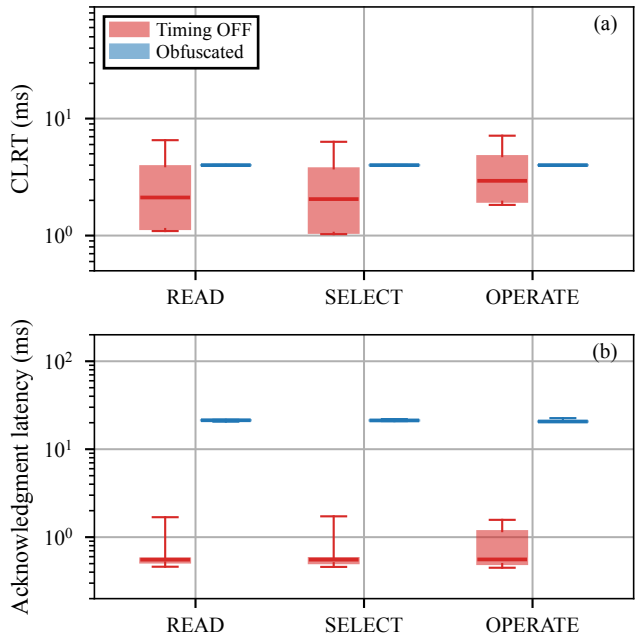


Fig. 5. **CLRT and acknowledgment latency** per transaction class and arm, pooled over 22 sessions. Boxes span the quartiles; whiskers the 5th to 95th percentile.

Result. Returning to Figure 5, the OPERATE median falls from 2.937 ms with the mechanism off to 4.000 ms with it on, and the interquartile range falls from 2.750 ms to 0.0058 ms. The third panel of Figure 7 shows the same value in every session. Because the codebook draws the relay-facing hold per transaction over $\{2, 6, 12\}$ ms while the master-visible interval does not move, the observable is independent of that hold, which the control plane sustains by admitting only offsets that exceed the largest hold plus the relay's own response time.

The anchors are visible in the data. The acknowledgment latency in Figure 5 settles at 20.650 ms for OPERATE against 21.336 ms for READ and 21.239 ms for SELECT. That 0.686 ms difference is what anchoring the control path to the request and the read path to the acknowledgment predicts, so the measurement confirms which rule each transaction took. It is also the one timing quantity the framework leaves separable, and Section VI-F measures what an observer can do with it.

F. Effectiveness in RO3: Transaction-Class Timing Leakage

Why. RO3 asks how much the timing tells an observer about the transaction class, and whether the mechanism removes it. We treat this as a three-class problem over READ, SELECT and OPERATE, for which the chance balanced accuracy is one third.

Result. In Figure 8, we show the balanced accuracy of a random forest and the mutual information between the CLRT and the class, each computed on the held-out session and averaged over the 22 folds. Using the CLRT alone, balanced accuracy falls from 0.651 with the mechanism off to 0.333 with it on, which is chance to three decimal places and with no variation across folds, and the mutual information falls from 0.266 bits to 0.007 bits. Consequently the feature that Formby et al. use to separate operations no longer carries the class.

What survives. When the acknowledgment latency and the response time are added to the feature set, balanced accuracy under the mechanism recovers to 0.655. Panels (c) and (d) of Figure 8 show why. With the mechanism on and the CLRT alone, the classifier assigns almost every transaction to one class, which is the chance behaviour the accuracy reports. With the full feature set, OPERATE is recovered while READ and SELECT stay confused with each other. This is because the control path settles 0.686 ms before the read path, a gap seven times the 0.097 ms that separates READ from SELECT. The cross-layer response time, the feature Formby et al. rely on, is therefore removed as a class signal, and what remains is the difference between the two anchors, which a common anchor would close.

G. Cost

Result. In Figure 9, we show what the mechanism costs. The median response time rises from 2.680 ms to 25.337 ms for READ, from 2.622 ms to 25.239 ms for SELECT and from 3.465 ms to 24.650 ms for OPERATE, so the mechanism adds about 21 to 23 ms to a transaction. That delay is the mechanism rather than an overhead of it, since the release schedule is what removes the feature. Every one of the 132 captures carries exactly 1,448 frames and 130,708 bytes for 440 transactions, in both arms, so with the size carve disabled the framework adds no packet and no byte and only moves packets in time. Because a master that polls sequentially waits for each response, the achievable rate falls from roughly 370 to roughly 40 transactions per second; pipelined polling is not measured here.

H. Limitations

The results are master-facing timing from one SEL-751A relay behind one switch. We do not instrument the relay-facing link, so the configured hold J and the release toward the relay are established from the configuration rather than measured. The framework changes when the wire carries the outstation's answer and not what the answer says, so the DNP3 function codes stay visible and the results concern the timing channel alone; the classifier separates transaction classes on one device and is not a device-identification experiment. Two behaviours follow from holding a transaction and are worth naming: the program suppresses a retransmitted response while a transaction is pending, and drops a retransmitted OPERATE once the epoch has been released. Both are deliberate properties of the release schedule.

VII. Related Work

Device fingerprinting. Devices can be identified from what they emit, even without any access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock as seen in TCP timestamps [15], and GTID fingerprints a device and its type from the timing signature of its packets [16]. These

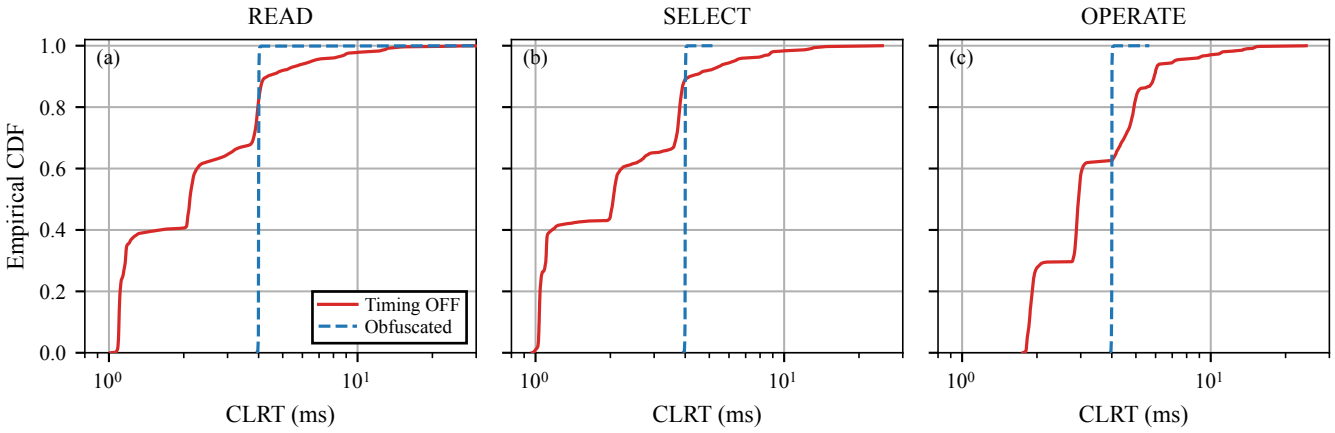


Fig. 6. **Empirical CDF of CLRT** per transaction class, pooled over 22 sessions. The abscissa is logarithmic.

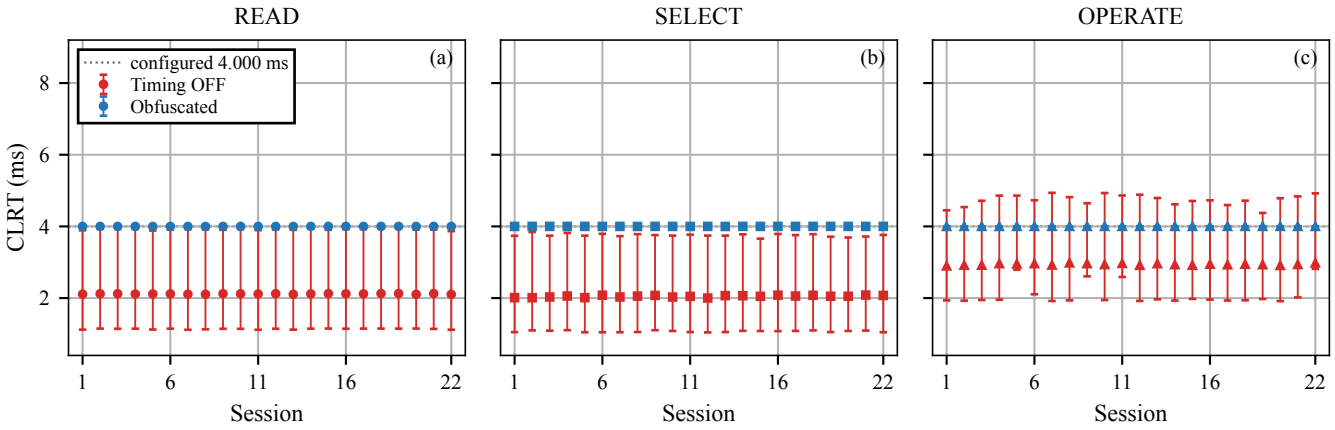


Fig. 7. **Per-session CLRT across 22 sessions**, one panel per transaction class. Markers are session medians; bars span the interquartile range.

works define the threat we address; our objective, on the other hand, is to remove the timing features they rely on from the master-facing observation.

ICS and power-system fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time, measured at the device, of DNP3 outstations [10], and Gu et al. added device physics as a feature [17]. Jeon et al. fingerprint SCADA devices passively without deep packet inspection [18]. The reconnaissance these techniques serve enables false-data injection and process-control attacks [14], [19]. Defenses in this setting have so far worked at the content and connectivity layer: DefRec virtualizes physical functions to present deceptive content [20], RAINCOAT randomizes data acquisition and injects decoy measurements [21], and a companion testbed evaluates such anti-reconnaissance approaches on grid infrastructure [22]. Compared to these, our framework works below the semantics they reshape and complements them: it changes when the wire carries the outstation's answer, not what the answer says.

General traffic obfuscation. Traffic morphing transforms one class's packet-size distribution into another's [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [6], and WTF-PAD pads inter-packet gaps adaptively [23]. However, deep fingerprinting attacks defeat several of these [5], and their accuracy at Internet scale is contested [24]. The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [25], and Pacer and NetShaper shape a tenant's traffic to a schedule that bounds side-channel leakage in the cloud [26], [27]. These defenses assume an encrypted payload and a cooperating end host or hypervisor that can add cover traffic. Our framework, in contrast, targets a plaintext protocol whose feature is one interval inside a request-response exchange, adds no byte, and runs where the outstation cannot be changed.

Programmable data-plane traffic transformation. P4 defines the programmable parser and match-action model we use [12]. PIFO defines programmable scheduling at line

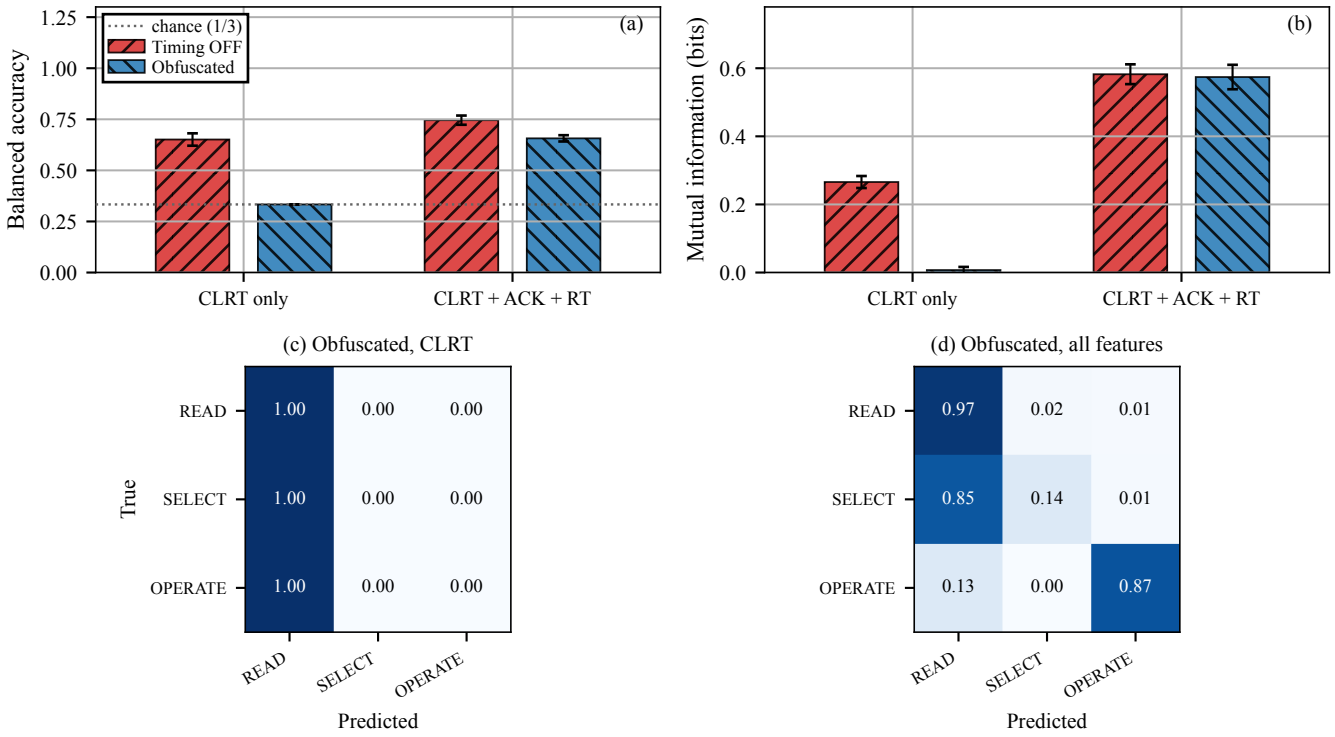


Fig. 8. **Transaction-class leakage under session-disjoint evaluation.** (a) balanced accuracy, (b) mutual information, over 22 leave-one-session-out folds; (c, d) row-normalised confusion under the mechanism, for the two feature sets.

rate [28], and SP-PIFO approximates it with strict-priority queues on today's switches [29], which is the primitive our reservoirs use. Ditto pads and injects chaff to a fixed pattern at line rate [7], Minos morphs and schedules encrypted traffic on a programmable switch with low overhead [8], and Securitas mixes fragmentation and insertion under a learned policy across several data planes [9]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [30]. Closest to our setting, Hu et al. use programmable switches to enhance industrial-protocol security [31]. Our framework differs from these in the observable it addresses, i.e., the master-visible timing of a DNP3 transaction, and in holding packets rather than adding or reshaping them.

DNP3 timing and security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [13], and specification-based and state-based intrusion detection has been adapted to DNP3 [32], [33]. These do not change the timing an observer sees. The physical operation-time feature we address on the control path is the one Formby et al. measured on a physical outstation [10]; we report what the master sees under the defense and, unlike a measurement of the device, we do not observe the relay-facing side.

VIII. Conclusion

A passive observer of legacy DNP3 traffic can fingerprint an outstation from the timing of its responses, and the size- and delay-based obfuscation built for encrypted web traffic does not transfer to a plaintext control protocol on devices that cannot be changed. In this paper, we presented an in-network timing obfuscation framework that runs in the data plane of a single programmable switch. It holds the packets that carry an outstation's timing and releases them at configured offsets, with one rule for polls and the SELECT phase of control that is anchored to the outstation's acknowledgment, and another rule for OPERATE that is anchored to the request. We implemented it as one P4 program on a Tofino-1 and evaluated it against a physical SEL-751A relay. Over 22 independent sessions and 63,360 transactions, the framework pinned the CLRT of READ, SELECT and OPERATE to 4.000 ms with an interquartile range of 0.006 ms, where the relay's own median CLRT was 2.116, 2.050 and 2.937 ms with an interquartile range near 2.7 ms; it held the master-visible OPERATE interval at that value across configured relay-facing holds of 2, 6 and 12 ms; and, evaluated on sessions the classifier never saw, it reduced the balanced accuracy of a three-class transaction classifier from 0.651 to chance and the mutual information between the class and the CLRT from 0.266 bits to 0.007 bits. It costs about 21 to 23 ms per transaction and no additional packet or byte, and its two offsets set the delay and the visible interval independently. These results show the suppression of one timing feature on one relay; the function codes stay visible, the relay-facing side was not observed, and an observer who also measures the acknowledgment latency can still separate the control path from the read path. As the

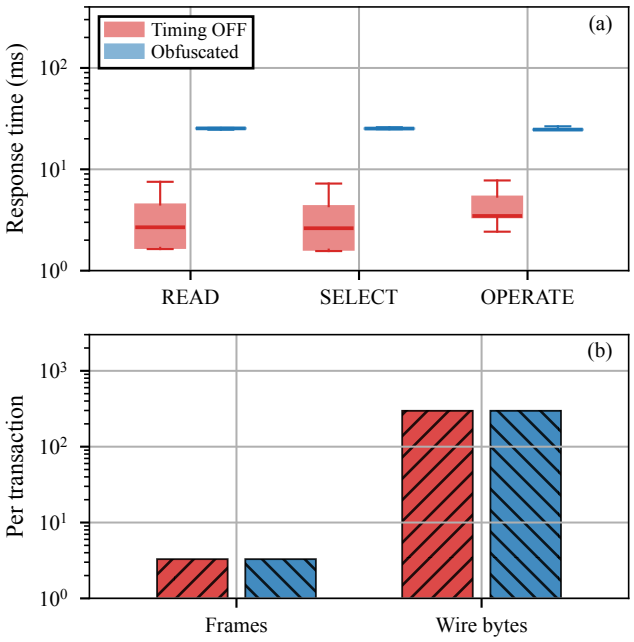


Fig. 9. **Cost of the mechanism:** (a) response time per transaction class, (b) frames and bytes per transaction. Both ordinates are logarithmic.

next validation, we plan a relay-facing tap, so that the release at $T_0 + J$ is measured rather than configured, and a shared anchor for the two paths, so that the residual difference is removed rather than reported.

References

[1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the Cyber Attack on the Ukrainian Power Grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016.

[2] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail," in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012.

[5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.

[6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.

[7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN Traffic Obfuscation at Line Rate," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.

[8] Z. Wang, Q. Li, G. Xie, D. Zhao, K. Li, Z. Fan, L. Ma, and Y. Jiang, "Minos: A Lightweight and Dynamic Defense against Traffic Analysis in Programmable Data Planes," in *Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*, 2025.

[9] G. Xie, Q. Li, Z. Shi, G. Antichi, Y. Zhu, K. Li, C. Weng, S. Miano, Y. Jiang, and M. Xu, "Defending against Traffic Analysis Attacks with Flexible In-Network Obfuscation," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, pp. 2043–2063.

[10] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016.

[11] IEEE, "IEEE Standard for Electric Power Systems Communications—Distributed Network Protocol (DNP3)," 2012.

[12] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[13] S. East, J. Butts, M. Papa, and S. Shenoi, "A Taxonomy of Attacks on the DNP3 Protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol.\ 311)*. Springer, 2009, pp. 67–81.

[14] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks against State Estimation in Electric Power Grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.

[15] T. Kohno, A. Broido, and K. C. Claffy, "Remote Physical Device Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.

[16] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A Technique for Physical Device and Device Type Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.

[17] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for Cyber-Physical System Security: Device Physics Matters Too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, Sep. 2018.

[18] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive Fingerprinting of SCADA in Critical Infrastructure Network without Deep Packet Inspection," arXiv:1608.07679 [cs.CR], Aug. 2016.

[19] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks Against Process Control Systems: Risk Assessment, Detection, and Response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.

[20] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing Physical Function Virtualization to Disrupt Reconnaissance of Power Grids' Cyber-Physical Infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.

[21] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of Network Communication in Power Grid Cyber Infrastructure to Mislead Attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.

[22] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-Physical Testbed: Case Study to Evaluate Anti-Reconnaissance Approaches on Power Grids' Cyber-Physical Infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.

[23] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an Efficient Website Fingerprinting Defense," in *Computer Security – ESORICS 2016*, 2016.

[24] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website Fingerprinting at Internet Scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[25] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the Smart Home Private with Smart(er) IoT Traffic Shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019.

[26] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud," in *Proceedings of the 31st USENIX Security Symposium*, 2022.

[27] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A Differentially Private Network Side-Channel Mitigation System," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[28] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, Agrawal, Anurag, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable Packet Scheduling at Line Rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.

[29] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating Push-In First-Out Behaviors using Strict-Priority Queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.

[30] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and Practical Network Topology Obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.

[31] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial Network Protocol Security Enhancement Using Programmable Switches," in *Proceedings of the IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.

[32] H. Lin, A. Slagell, C. D. Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a Specification-Based Intrusion Detection System for the DNP3 Protocol," in *Proceedings of the Eighth Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.

[33] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 State-Based Intrusion Detection System," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.